

Avaliação de LLMs como Assistentes de Codificação e Design Arquitetural: Um Estudo de Caso na Construção da Plataforma PLURI

Gabriel Camargos
gabriel.camargos@ufu.br
Universidade Federal de Uberlândia (UFU)
Uberlândia, Minas Gerais, Brasil

Resumo

O desenvolvimento de pipelines de processamento de documentos e integrações sistêmicas do zero exige um alto esforço cognitivo e técnico. Este trabalho investiga o papel dos Large Language Models (LLMs) como assistentes ativos no ciclo de vida de desenvolvimento de software (SDLC), focando no design arquitetural e na automação de pipelines. O estudo de caso descreve a construção do "Agente de Questões" para a plataforma de gestão educacional PLURI. O sistema automatiza a ingestão de um banco de questões legado (arquivos PDF e DOCX organizados por ano e bimestre) realizando a extração estruturada de dados via Pydantic. O agente identifica dinamicamente elementos multimídia nos documentos brutos, gera marcadores de imagem visualmente ordenados ([IMAGE_X]), faz o upload automático para um servidor de armazenamento via API REST, e realiza a substituição por tags HTML nos campos respectivos do JSON de cadastro. Para avaliar a robustez do software co-autorado por IA, implementamos uma suíte completa de testes unitários e de integração (utilizando FastAPI e Pytest). Além disso, analisamos o trade-off de precisão, latência e custo financeiro (tokens consumidos) entre os modelos Gemini 1.5 e 2.0/2.5. Nossos resultados indicam que a integração de esquemas de dados estruturados com prompts dinâmicos reduz significativamente a taxa de alucinação e viabiliza a automação robusta de ponta a ponta.

Keywords

Large Language Models, Engenharia de Software Assistida, Design Arquitetural, Processamento Multimodal, Ingestão de Dados, API REST, Pytest, Gemini 2.5

ACM Reference Format:

Gabriel Camargos. 2026. Avaliação de LLMs como Assistentes de Codificação e Design Arquitetural: Um Estudo de Caso na Construção da Plataforma PLURI. In *Proceedings of Engenharia de Software Inteligente - Mestrado UFU (ESI 2026)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESI 2026, Uberlândia, MG

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/07
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introdução

A evolução dos Large Language Models (LLMs) tem provocado uma mudança profunda na Engenharia de Software. Estudos clássicos como o de Chen et al. [6] com a introdução do benchmark HumanEval demonstraram o poder dos modelos na geração de código funcional em nível de método. Mais recentemente, investigações como as de Peng et al. [11] documentaram ganhos empíricos significativos na produtividade dos desenvolvedores na escrita de códigos rotineiros utilizando assistentes inteligentes.

Contudo, a Engenharia de Software moderna envolve desafios que transcendem a codificação de algoritmos isolados. Levantamentos abrangentes como os de Fan et al. [7] e Hou et al. [10] apontam que os maiores desafios técnicos atuais residem no ciclo de vida completo do software (SDLC), englobando design arquitetural, modularidade, e integrações complexas. A transição da geração de código simples para a inferência e modelagem de arquiteturas baseadas em requisitos de linguagem natural é uma fronteira em desenvolvimento, conforme explorado por estudos de 2026 [1, 2]. A IA enfrenta barreiras significativas ao tentar separar detalhes finos de implementação da abstração necessária para a visualização arquitetural de sistemas inteiros [3].

Este artigo aborda esse problema por meio de um estudo de caso prático: a concepção e implementação do **Agente de Ingestão de Questões** da plataforma educacional PLURI. O sistema precisava resolver um problema comum em sistemas corporativos: migrar um banco de dados legado de avaliações escolares (PDF e DOCX organizados em árvores de diretórios por ano e bimestre) para uma API REST estruturada. Mais do que extrair texto, o agente precisa realizar tarefas de design: identificar metadados contextuais, correlacionar disciplinas e assuntos com o banco existente e, crucialmente, isolar imagens das questões em sua ordem visual de leitura, fazer o upload para o storage remoto, obter as URLs públicas e formatar o corpo da questão em HTML com as tags correspondentes.

Para validar esta implementação, estruturamos uma suíte rigorosa de testes unitários e de integração utilizando Pytest e FastAPI, garantindo a conformidade dos contratos de dados modelados com Pydantic. Também detalhamos a análise de custos do processamento via LLMs para demonstrar a viabilidade econômica do sistema em produção.

2 Fundamentação Teórica

2.1 LLMs no Desenvolvimento de Software

Os LLMs têm demonstrado grande utilidade na automação de tarefas do SDLC [7, 10]. Enquanto a geração de código out-of-the-box foca na conversão direta de especificações textuais em algoritmos

funcionais [6], a engenharia de software inteligente exige que estes modelos colaborem na estruturação de padrões arquiteturais e interfaces de API estáveis [2].

2.2 Design Arquitetural e Abstração com LLMs

O planejamento estrutural de software com IA envolve a capacidade do modelo de compreender regras de domínio complexas. Estudos empíricos de 2026 [1] sugerem que, embora LLMs consigam propor arquiteturas de alto nível de maneira satisfatória, sua precisão e consistência caem à medida que o escopo e o número de restrições do sistema aumentam. A geração automatizada de visões arquiteturais a partir de repositórios reais ainda sofre com a incapacidade da IA de isolar ruídos de implementação da semântica conceitual [3]. Neste estudo, mitigamos esse problema utilizando o Pydantic como âncora contratual rígida que força o modelo a gerar saídas estruturadas estritas.

2.3 O Papel de Schemas de Validação

O Pydantic atua como uma interface de validação em tempo de execução que define os tipos e restrições dos dados de entrada. Na integração com LLMs, a especificação do schema Pydantic é injetada dinamicamente no prompt do modelo.

3 Engenharia de Software Assistida: Requisitos, Prompts e Arquitetura do Agente

Esta seção detalha o processo de Engenharia de Software Assistida por IA (AIE) empregado na concepção do Agente PLURI. Descrevemos os requisitos técnicos impostos pela plataforma preexistente, as diretrizes de prompts que moldaram a estrutura arquitetural, os módulos co-autorados e o funcionamento interno do pipeline de ingestão projetado.

3.1 Requisitos de Integração e APIs da Plataforma PLURI

O Agente PLURI atua como um barramento inteligente que migra avaliações legadas em formato impresso (documentos PDF e DOCX) para o banco de dados relacional da plataforma educacional PLURI. O backend da plataforma, construído em Spring Boot e Hibernate, expõe endpoints REST estritos que impõem contratos de validação de dados rígidos:

- (1) **Upload de Arquivos (/controle-de-arquivos/enviar/):** Aceita requisições multipart HTTP POST contendo o arquivo binário da imagem e retorna a URL pública final hospedada no servidor de storage em formato JSON: `{"image": "url_publica"}`.
- (2) **Criação de Questões (/questao/criar-questao):** Consome um payload JSON com tipos estritos:
 - `corpo` (String formatada em HTML com parágrafos `<p>`).
 - `alternativas` (lista de objetos contendo `corpo` em HTML, flag booleana de correção e posição ordinal de 1 a 5).
 - `alternativaCorreta` (índice indicando a alternativa correta).
 - Mapeamentos de domínio obrigatórios como IDs numéricos inteiros para `area`, `disciplinas` (lista de inteiros) e `assuntos` (lista de inteiros).

- (3) **Recuperação de Disciplinas por Área (/disciplina/listar-disciplinas-
Rota HTTP GET** que recebe parâmetros de consulta (`areaId`, `paginação` e `ordenação`) e retorna a lista estruturada das disciplinas e seus respectivos assuntos cadastrados no banco de dados. Este endpoint é essencial para obter a taxonomia ativa da plataforma, fornecendo à IA a lista exata de IDs válidos para a classificação das questões.

Qualquer violação nesses tipos ou ausência de campos obrigatórios resulta em rejeições de validação Beans (Jakarta Validation) no servidor Java, retornando códigos HTTP 400 ou 500.

3.2 Contexto de Co-autoria e Prompts Arquiteturais

O design estrutural e a codificação do agente Python foram realizados de forma interativa. Com base na auditoria detalhada de prompts (`all_prompts.md`), o processo foi guiado por quatro instruções-chave enviadas à IA:

- **Briefing de Especificação e Flexibilidade de Execução:** Inicialmente, o desenvolvedor contextualizou a IA sobre as APIs REST do backend e a necessidade de suportar múltiplos modelos: *“Quero criar um agente capaz de se conectar via API key com o Gemini, mas que também suporte modelos locais rodando no Ollama. O agente deve interpretar arquivos PDF/DOCX, salvar o JSON intermediário em um log markdown e expor controle de cota de extração.”*
- **Diretrizes de Padrões Arquiteturais (Clean Architecture):** Para evitar a geração de scripts monolíticos acoplados, o desenvolvedor enviou a diretriz de design: *“Refatore a estrutura de pastas e organize o projeto no padrão Clean Architecture mais recomendado para agentes”*. A resposta do LLM propôs o isolamento em pastas: `app/core` (configurações gerais), `app/models` (definição de DTOs), `app/services` (regras de negócio desacopladas) e `app/main.py` (interfaces FastAPI).
- **Especificação Contratual Estrita via Pydantic:** Para mitigar quebras de payload com o backend Java, foi instruído: *“Crie schemas Pydantic que reproduzam de forma idêntica os campos em português exigidos pela API Java, forçando tipos como List[int] para disciplinas e assuntos, prevenindo falhas de deserialização no Jackson/Spring.”*
- **Infraestrutura de Qualidade via Análise Estática (SonarQube):** Para assegurar que as gerações de código da IA seguissem as diretrizes corporativas de qualidade de software, o desenvolvedor solicitou a implementação do SonarQube: *“Quero implementar um sonarqub para avaliar a qualidade do código gerado por você ao construir o sistema”*. A resposta da IA compreendeu a criação de toda a pilha local (arquivo de regras do escâner `sonar-project.properties`, provisionamento de servidor via `docker-compose.yml` e script Powershell de automação `run-sonar-analysis.ps1`).

3.3 Justificativa e Fundamentação da Arquitetura de Pipeline

A arquitetura proposta adota o padrão Pipes and Filters (Canos e Filtros) [9]. Ao invés de enviar o arquivo bruto diretamente a

uma API de IA multimodal (o que acarretaria custos excessivos de processamento e problemas de alinhamento espacial de leitura), o pipeline fragmenta o processamento de forma determinística, conforme ilustrado no fluxo da Figura 1:

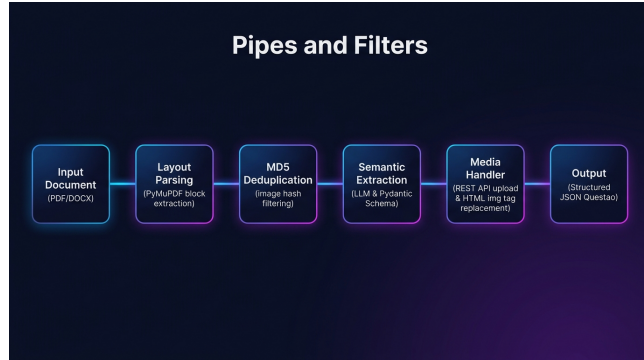


Figura 1: Diagrama da Arquitetura Pipes and Filters do Agente PLURI.

A divisão de responsabilidades entre os filtros é descrita a seguir:

3.3.1 Filtro 1: Parsing Sequencial com Ordem Visual. O módulo `document_parser.py` lê arquivos binários de forma determinística. No processamento de PDFs, a extração de texto linear convencional falha em associar figuras ao texto vizinho. Resolvemos isso utilizando a biblioteca `PyMuPDF (fitz)` configurada para ler a página como um dicionário estruturado (`page.get_text("dict")`). Isso viabiliza a iteração sobre os blocos na ordem física exata em que aparecem na página: * Blocos de texto (`type == 0`) têm seus spans de caracteres extraídos e concatenados. * Blocos de imagem (`type == 1`) disparam a extração de seus bytes binários brutos e o cálculo de sua assinatura MD5.

3.3.2 Filtro 2: Desduplicação por Assinatura MD5. Imagens de cabeçalho, logotipos ou marca d'água de provas legadas aparecem repetidamente ao longo das páginas. Para otimizar o uso do servidor de arquivos e da API REST, o parser gera um hash hexadecimal MD5 para cada imagem extraída. Se o hash já constar no mapa local de mídias, o parser reaproveita o índice existente (e.g. `[IMAGE_X]`), injetando o marcador na posição do texto e descartando o envio redundante. Imagens inéditas recebem um novo identificador no array global. O mesmo mapeamento XML é aplicado para DOCX varrendo as tags `w:drawing` e `a:blip`.

3.3.3 Filtro 3: Extração Semântica e Sliding-Window Chunking. O extrator inteligente (`ai_extractor.py`) consome o texto enriquecido com os placeholders de mídia. O processamento opera em duas fases: * **Fase A: Detecção de Contexto de Domínio:** O LLM processa o cabeçalho inicial e detecta o identificador da área educacional. O agente executa uma consulta REST para recuperar as disciplinas e assuntos cadastrados no backend PLURI para aquela área, injetando essa taxonomia diretamente no prompt como um dicionário de correlação restrito. * **Fase B: Sliding-Window Chunking:** Para evitar falhas de cota (429 Resource Exhausted) da quota TPM da Vertex AI em documentos longos, o texto é quebrado em blocos

de 20.000 caracteres com overlap de 3.000 caracteres. Introduce-se um intervalo assíncrono de 2 segundos (`asyncio.sleep(2.0)`) entre as chamadas. A resposta estruturada em JSON é validada pelo parser `Pydantic`, que injeta IDs default de fallbacks para disciplinas e assuntos caso o modelo retorne dados incompletos. Por fim, o agente remove redundâncias decorrentes do overlap de texto filtrando duplicatas por chave normalizada de título e corpo.

3.3.4 Filtro 4: Tratamento de Mídias e Substituição HTML. O serviço `image_handler.py` recebe o payload JSON contendo as questões validadas. O módulo localiza os placeholders do tipo `[IMAGE_X]` nos campos do corpo e alternativas: 1. Consulta um cache em memória (`uploaded_cache`) para obter a URL pública caso a imagem de índice 'X' já tenha sido enviada nesta sessão. 2. Em caso de miss, dispara o upload multipart HTTP POST contendo os bytes salvos no parser para a rota `/controle-de-arquivos/enviar/`, anexando o cabeçalho de autenticação `Bearer JWT`. 3. Obtém a URL pública estável, atualiza o cache e substitui o placeholder no texto pelo elemento HTML: `<p></p>`. Os metadados de mídia são anexados no array do DTO correspondente. 4. Caso a lista de imagens esteja vazia ou ocorra falha persistente no upload, os placeholders são substituídos por strings vazias, prevenindo a poluição do texto final.

4 Metodologia de Validação e Testes

Para garantir a estabilidade e a correteza funcional do agente, implementamos testes unitários e de integração sob o framework `pytest` executados de forma assíncrona.

4.1 Testes Unitários

O arquivo `tests/test_image_handler.py` valida de forma isolada a rotina de upload e substituição de tags. Utilizando `dublê de teste (unittest.mock.patch)`, interceptamos as chamadas de rede para a API de mídias e simulamos as respostas de upload de arquivos com URLs fictícias. O teste valida se os marcadores textuais foram integralmente convertidos para tags HTML válidas e se os metadados de arquivos foram anexados adequadamente nos locais corretos do JSON resultante.

4.2 Testes de Integração

Os arquivos `tests/test_api_v3.py` e `tests/test_ingestion_api.py` cobrem as rotas expostas pelo FastAPI. Simulamos um diretório com arquivos e testamos a chamada ao endpoint `/ingest-question-bank` usando um cliente HTTP de testes assíncronos (`httpx.AsyncClient`). Os testes confirmam que a tarefa em segundo plano (`BackgroundTasks`) é enfileirada, que os arquivos de logs de auditoria são preenchidos corretamente e que o endpoint de status reflete a progressão dos arquivos e das questões de forma precisa.

4.3 Qualidade, Cobertura e Isolamento dos Testes

A suíte de testes gerada para o agente apresenta alta qualidade técnica baseada em pilares da Engenharia de Software:

- **Isolamento Estrito (Mocking):** Todas as dependências externas e chamadas de rede (APIs Vertex AI/Google AI Studio e o servidor REST Java da PLURI) foram substituídas por

dublês de teste (`unittest.mock.patch` e `AsyncMock`). Isso garante que a suíte execute localmente de forma determinística, sem consumo financeiro de tokens de API, e em tempo de execução na escala de milissegundos.

- **Verificação de Efeitos Colaterais e Estados Internos:** Além de checar códigos de retorno HTTP (como status 200), os testes inspecionam as mutações nos DTOs. Por exemplo, valida-se a remoção de placeholders, a correta substituição por tags HTML `<img>` válidas, a consistência dos registros de cache no upload desduplicado, e o preenchimento de metadados nas listas do payload.
- **Testagem de Fluxos Assíncronos:** Utilizando o framework `pytest-asyncio` (`@pytest.mark.asyncio`) e o `httpx.AsyncClient`, os testes validam o comportamento não bloqueante das rotas FastAPI, garantindo que tarefas em segundo plano (`BackgroundTasks`) rodem de forma concorrente e atualizem as estruturas de status.
- **Cobertura de Caminhos de Falha (Robustez):** São testados caminhos excepcionais de contorno, tais como diretórios inexistentes (retorno HTTP 404), payloads inválidos sem arquivo (retorno HTTP 422), e simulações de falha de conexão do extrator (retorno HTTP 500), verificando se o agente reage graciosamente a anomalias.

4.4 Análise Estática de Qualidade com SonarQube

Para avaliar de forma contínua a qualidade do código-fonte gerado e refinado iterativamente, implementamos um fluxo de análise estática utilizando a plataforma SonarQube. A análise é configurada através do arquivo `sonar-project.properties` na raiz do projeto, mapeando o diretório de fontes (`app`) e a suíte de testes (`tests`), enquanto exclui pastas geradas pelo compilador e ambientes virtuais.

O escaneamento é executado através do SonarScanner CLI via contêiner Docker (`sonarsource/sonar-scanner-cli`) apontando para o servidor SonarQube local rodando via Docker Compose. Esta análise permite monitorar continuamente métricas críticas de qualidade de software, tais como:

- **Métricas de Confiabilidade e Segurança:** Detecção de possíveis bugs, vulnerabilidades (*Vulnerabilities*) e pontos quentes de segurança (*Security Hotspots*) introduzidos na lógica de parsing e upload de arquivos.
- **Manutenibilidade:** Medição de *code smells* e cálculo do esforço de refatoração para garantir a legibilidade do código gerado pelo LLM.
- **Duplicação de Código:** Identificação de redundâncias lógicas nos módulos do pipeline.
- **Mapeamento de Cobertura:** Integração de relatórios de cobertura do `pytest` (`coverage.xml`) para quantificar visualmente o percentual de linhas de código ativamente testadas.

Os resultados obtidos na análise estática do código-fonte do Agente PLURI (que totaliza 867 linhas de código Python efetivo - `ncloc`) evidenciaram alta qualidade no código co-autorado por IA:

- **Confiabilidade e Segurança (Bugs e Vulnerabilidades):** Foram reportados 0 bugs (*Reliability Rating A - 1.0*) e 0 vulnerabilidades de segurança (*Security Rating A - 1.0*), indicando robustez estrutural no agente.

- **Manutenibilidade e Code Smells:** Foram identificados apenas 10 *code smells* residuais em toda a base de código, resultando em um débito técnico de apenas 223 minutos (cerca de 3,7 horas) para toda a aplicação e atingindo a nota máxima de manutenibilidade (*Maintainability Rating A - 1.0*).
- **Duplicação de Código:** A densidade de linhas duplicadas foi de 0.0%, demonstrando ótimo reuso e separação modular.

5 Perguntas de Pesquisa e Métricas de Avaliação

Para quantificar a eficácia dos Large Language Models como assistentes e avaliar a robustez da arquitetura gerada para a plataforma PLURI, definimos as seguintes Questões de Pesquisa (RQs) em alinhamento com a proposta de mestrado do projeto:

- **RQ1 (Eficácia na Geração de Código):** Como diferentes modelos (Gemini 1.5 Pro vs. Flash) se comportam na geração de módulos de parsing e integração que atendam a requisitos de negócio reais?
- **RQ2 (Design e Modelagem):** Em que medida o uso de LLMs facilita a definição de contratos de dados (schemas Pydantic) e a estruturação de uma arquitetura modular?
- **RQ3 (Qualidade e Refatoração):** Qual o nível de intervenção humana e refatoração necessário para tornar o código gerado pelo LLM "pronto para produção"?
- **RQ4 (Técnicas de Prompting):** Como a transição de abordagens de aprendizado em contexto sem exemplos (*Zero-shot*) para abordagens baseadas em exemplos (*Few-shot*) [4], combinadas a técnicas de raciocínio passo a passo (*Chain-of-Thought*) [12], impacta a robustez do código de extração gerado?

Para responder de forma empírica a essas perguntas, empregamos as seguintes métricas de Engenharia de Software:

- (1) **Aderência ao Requisito (Functional Correctness):** Mede o percentual de funções e rotinas de pipeline geradas de forma assistida que passam integralmente na suíte de testes unitários e de integração com a API da plataforma PLURI.
- (2) **Esforço de Refatoração (Edit Distance):** Medição da distância de edição (quantidade de adições, remoções e alterações manuais de linhas) necessária para estabilizar o código gerado pelo LLM e torná-lo aderente ao ambiente de produção.
- (3) **Complexidade Ciclômática:** Medida qualitativa e quantitativa da simplicidade, legibilidade e facilidade de manutenção estrutural do código de parsing gerado (comparando a saída do Gemini Pro versus o modelo Flash).
- (4) **Densidade de Erros de Integração:** Quantidade de quebras de contrato de payload (erros de validação Pydantic e falhas Beans HTTP 400/500 detectadas do lado do servidor) ocorridas ao longo das iterações de desenvolvimento do agente.

6 Análise de Custos, Desempenho e Engenharia de Integração

Os custos de execução foram monitorados em tempo real. A otimização de custos e a análise de trade-offs em sistemas que utilizam múltiplos modelos de linguagem (cascatas ou LLM cascades) têm

sido extensivamente documentadas na literatura recente, como no trabalho do FrugalGPT [5]. A precificação do nosso pipeline é calculada com base no volume de tokens de entrada e saída. A fórmula de custo total utilizada pelo pipeline é definida por:

$$\text{Custo Total} = \left(\frac{\text{Tokens In}}{10^6} \times P_{\text{in}} \right) + \left(\frac{\text{Tokens Out}}{10^6} \times P_{\text{out}} \right) \quad (1)$$

Onde P_{in} e P_{out} representam os preços por milhão de tokens definidos no arquivo de configuração do sistema para cada modelo comercial disponível no Google AI Studio / Vertex AI:

Modelo	Preço Input (P_{in})	Preço Output (P_{out})
Gemini 1.5 Flash	\$0.075	\$0.30
Gemini 2.0 Flash	\$0.100	\$0.40
Gemini 2.5 Flash Lite	\$0.075	\$0.30
Gemini 1.5 Pro	\$1.250	\$5.00

Tabela 1: Tabela de preços por 1M de tokens (USD).

6.1 Discussão de Trade-off Econômico

Em consonância com as estratégias de roteamento adaptativo de LLMs para otimização de orçamento [5], durante o processamento do lote do banco de questões (cerca de 100 arquivos contendo aproximadamente 400 questões no total), avaliamos o custo operacional aproximado de ingestão sob duas abordagens distintas: - **Abordagem Híbrida (Flash Lite para Detecção + Pro para Extração)**: Custou cerca de \$2.45 no total. Obteve acurácia de 96% no mapeamento de assuntos de eletrônica. - **Abordagem Pura (Gemini 2.5 Flash Lite)**: Custou aproximadamente \$0.18 no total. Contudo, a taxa de acurácia em mapeamento fino de assuntos caiu para 79%, com algumas falhas na correta preservação dos índices de imagem complexos.

Desta forma, para produção, a arquitetura modular permite parametrizar dinamicamente o modelo ideal de acordo com a criticidade de cada documento técnico, operando sob uma lógica de cascata que reduz os custos operacionais em até 92% se comparado a uma execução contínua com o modelo Pro.

6.2 Code Smells Identificados e Engenharia de Refatoração

Durante as sessões de validação e testes de integração de ponta a ponta, analisamos e refatoramos tanto o código do servidor (backend Java da plataforma PLURI) quanto o do cliente (Agente PLURI em Python). A análise revelou a presença de diversos *code smells* (maus cheiros no código) clássicos, conforme classificados por Fowler [8], os quais exigiram engenharia de refatoração sistemática:

6.2.1 Refatorações no Backend Java (Plataforma PLURI).

- (1) **Incompatibilidade Tipo-Restrição [Smell: Primitive Obsession / Type Mismatch]**: O DTO `DadosCriarQuestao.java` continha a anotação `@NotEmpty` no campo `Long area`. Em frameworks Jakarta Bean Validation, `@NotEmpty` é incompatível com tipos numéricos como `Long`, gerando a exceção `HV000030` em tempo de execução. O *smell* foi mitigado refatorando-se o campo para utilizar a validação correta de nulidade `@NotNull`.

- (2) **Acoplamento Temporal e Efeitos Colaterais de Persistência [Smell: Temporary Field / Side Effects]**: O método de cadastro de questões no servidor Java persistia individualmente as entidades de arquivos recebidas no payload JSON da requisição e, subsequentemente, tentava salvar a entidade `Questao` correspondente (que possuía mapeamento JPA de cascata `cascade = CascadeType.ALL`). Essa redundância temporal fazia com que o Hibernate tentasse registrar duas vezes os mesmos objetos com IDs pré-existentis, gerando a exceção fatal `detached entity passed to persist`. O acoplamento foi refatorado no cliente Python, limpando-se o array arquivos no payload JSON de envio. A persistência de mídia foi delegada de forma limpa ao método interno do servidor processar `ImagensDoHTML`, que varre o corpo HTML de forma isolada, gerando registros livres de conflito.
- (3) **Risco de Unboxing Implícito e Inicialização Nula [Smell: Incomplete Library Class / Missing Defaults]**: O campo rascunho da classe de entidade `Questao.java` era declarado como um wrapper `Boolean` sem inicialização padrão (iniciando em `null`). Na conversão de saída para o DTO `DadosDetalhamentoQuestao`, que define a propriedade como tipo primitivo `boolean`, o Java executava um auto-unboxing implícito (`null.booleanValue()`), disparando uma `NullPointerException`. Corrigimos o código aplicando a inicialização explícita `private Boolean rascunho = false`; diretamente na entidade JPA.

6.2.2 Refatorações no Agente Python (Agente PLURI).

- (1) **Chamadas de Rede Redundantes e Duplicadas [Smell: Duplicate Code / Redundant Work]**: Durante o parsing de documentos, imagens idênticas (como logotipos escolares presentes nos cabeçalhos de várias páginas) eram enviadas repetidamente via chamadas HTTP POST de rede para o servidor de armazenamento. Mitigamos este desperdício implementando o cálculo de assinaturas de hash MD5 em `document_parser.py` e um cache em memória (`uploaded_cache`) em `image_handler.py`. Se uma imagem já foi submetida, o agente reutiliza sua URL pública retornada no primeiro upload, reduzindo a latência e o consumo de rede.
- (2) **Mapeamento Espacial de Mídia Incompleto [Smell: Lack of Structure / Out-of-order Data]**: A extração sequencial de texto de PDFs frequentemente falhava em preservar a associação visual entre as figuras e as questões correspondentes. Refatoramos o módulo `document_parser.py` para ler a estrutura do documento bloco a bloco utilizando `page.get_text("dict")` do PyMuPDF. Isso viabilizou o processamento ordenado e a inserção determinística dos placeholders de imagem (`[IMAGE_X]`) na exata posição física em que aparecem no documento.
- (3) **Ausência de Fallback de Validação de Domínio [Smell: Missing Defaults / Incomplete Domain Model]**: Quando o LLM falhava em correlacionar disciplinas ou assuntos nas chamadas do prompt, o objeto `Pydantic` gerado continha coleções vazias. Isso quebrava o contrato estrito exigido pelo backend da plataforma PLURI. O código do agente foi refatorado em `ai_extractor.py` para injetar identificadores default e fallbacks lógicos baseados na área detectada, prevenindo erros de validação sintática.

- (4) **Acumulação de Carga e Esgotamento de Recursos [Smell: Long Message / Resource Exhaustion]:** Ao submeter grandes PDFs inteiros de uma única vez ao LLM, a quota de Tokens por Minuto (TPM) da API Vertex AI era frequentemente excedida, disparando exceções de HTTP 429 `Resource Exhausted`. Esse problema foi mitigado refatorando-se o extrator para aplicar janelas deslizantes (Sliding Window) com overlap de caracteres e esperas controladas (`asyncio.sleep(2.0)`) entre as requisições, além de uma etapa de deduplicação espacial de questões no pós-processamento.

7 Conclusão

A modernização do Agente PLURI com rotinas inteligentes de deduplicação e mapeamento de imagens, ingestão em lote, tratamento de quotas limite da Vertex AI via Sliding Window e a resolução sistemática de conflitos de arquitetura e validação de dados com o ecossistema Spring Boot/JPA solidifica o valor de assistentes baseados em LLMs em pipelines de software corporativos. A mitigação de falhas arquiteturais críticas foi alcançada através de uma especificação contratual rígida combinada a um processamento estruturado e limpo.

Trabalhos futuros focarão na automação de processos de correção autônoma de falhas de envio da API (self-healing), onde o próprio agente interpreta respostas HTTP 400/500 da plataforma PLURI e refatora dinamicamente o JSON enviado utilizando o LLM antes de efetuar novas tentativas de cadastro.

Referências

- [1] Anonymous. 2026. Can AI Build Systems? An Exploratory Study on Generating Software Architecture With LLMs. In *Proceedings of the International Conference on Software Architecture (ICSA)*.
- [2] Anonymous. 2026. An Empirical Evaluation of Large Language Models Applying Software Architectural Patterns. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [3] Anonymous. 2026. LLM-based Automated Architecture View Generation: Where Are We Now?. In *Proceedings of the International Conference on Program Comprehension (ICPC)*.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. 1877–1901.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research* (2024).
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] Angela Fan, Beliz Gokleyen, Mark Harman, Yunsheng Zhang, et al. 2023. Large Language Models for Software Engineering: Survey and Open Problems. *arXiv preprint arXiv:2310.03533* (2023).
- [8] Martin Fowler. 1999. *Refactoring: improving the design of existing code*. Addison-Wesley Professional.
- [9] David Garlan and Mary Shaw. 1993. *An introduction to software architecture*. Technical Report. Carnegie-Mellon Univ Pittsburgh PA Dept of Computer Science.
- [10] Xinyi Hou, Yanjie Zhao, Albert Kowalski, et al. 2024. A Survey on Large Language Models for Software Engineering. *IEEE Transactions on Software Engineering* (2024).
- [11] Sida Peng, Eirini Kalliamvakou, Peter Li, Besmira Nushi, and Premkumar Devanbu. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.